

Arbitrary Precision Arithmetic Library in Java

Akshat Banzal

Roll No.: [CS24BTECH11005]

May 2, 2025

Contents

1	Introduction	2
2	Design Overview	2
2.1	Class Design	2
2.1.1	AInteger	2
2.1.2	AFloat	2
3	How to Use the Library	2
3.1	As a CLI Tool	2
3.2	As a Java Library	2
4	Implementation Details	3
5	Build and Execution	3
5.1	Building the Project	3
5.2	Maven Targets	3
6	Testing and Verification	4
7	Limitations	4
8	Git Usage	4
9	Learnings	4

1 Introduction

This project involves building an arbitrary precision arithmetic library in Java. The goal is to support exact arithmetic for integers and floating-point numbers beyond the native Java limitations using string representations. This is achieved via two classes `AInteger` and `AFloat`, encapsulated in the `arbitraryarithmetic` package.

2 Design Overview

2.1 Class Design

2.1.1 `AInteger`

- Constructors: `AInteger()`, `AInteger(String)`, copy constructor.
- Static method: `parse(String)`.
- Operator overloading via: `add()`, `subtract()`, `multiply()`, `divide()`.
- Handles arbitrarily large integers using character-level operations.

2.1.2 `AFloat`

- Constructors: `AFloat()`, `AFloat(String)`, copy constructor.
- Static method: `parse(String)`.
- Operator overloading via: `add()`, `subtract()`, `multiply()`, `divide()`.
- Internally handles decimal precision and avoids floating point errors.

3 How to Use the Library

3.1 As a CLI Tool

```
java MyInfArith <int/float> <add/sub/mul/div> <operand1> <operand2>
```

3.2 As a Java Library

- Import `arbitraryarithmetic.AInteger` and/or `arbitraryarithmetic.AFloat`.
- Use constructors and arithmetic methods directly.

4 Implementation Details

- BigInteger and BigDecimal are not used. All computations are done on string representations.
- Helper methods handle digit-by-digit arithmetic operations.
- Floats are handled as two separate integer parts (integer and fractional).
- Division handles precision up to 30 decimal places as per requirement.

5 Build and Execution

5.1 Building the Project

Use the provided Python script or Ant/Maven configuration to compile and run:

```
python3 run.py int add 12345 67890
# or
ant run
# or
mvn compile exec:java
```

5.2 Maven Targets

- mvn clean compile - Compiles the project.
- mvn clean package - Packages.
- javac -cp .:(address of .jar file) (address of MyInfArith.java) - Links.

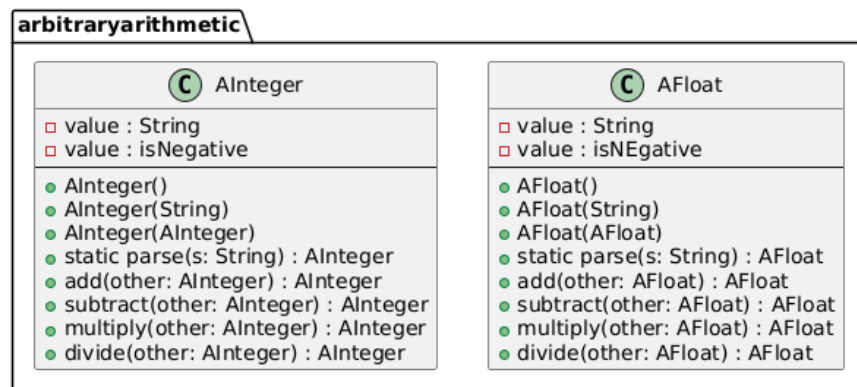


Figure 1: UML diagram of the project

6 Testing and Verification

All test cases provided in the problem statement have been executed successfully:

- Large integer and float operations.
- Division with up to 30 decimal digits.
- Handling of division by zero.
- Edge cases such as subtraction leading to negative numbers.

7 Limitations

- Currently assumes well-formed input strings.
- Negative floating-point number handling may have edge cases.
- Optimization for speed is limited; intended for correctness over performance.

8 Git Usage

- GitHub Repository: <https://github.com/akshat-31415/sdfProject1/>
- Git tags used for major milestones.
- README includes usage and build instructions.

9 Learnings

- Precision handling via string manipulation.
- Class design with proper OOP principles.
- Integration with Maven and use of CLI arguments in Java.
- Git workflows.